INDUSTRIAL INTERNSHIP REPORT ON

FILE ORGANIZER USING PYTHON

Prepared by

Kavya Kishor Jain

University Roll No. 24EJCCS186

Submitted in partial fulfilment for the award of the degree of

Bachelor of Technology

Computer Science and Engineering

Department of Computer Science and Engineering

Jaipur Engineering College & Research Centre

Jaipur, Rajasthan | 2026–27

EXECUTIVE SUMMARY

This report presents the File Organizer, a completed Python desktop application that sorts files in a selected folder into category subfolders based on file extension. It was developed as an Industrial Training project in the Department of Computer Science and Engineering at Jaipur Engineering College & Research Centre.

The application uses tkinter for its graphical interface, os and os.path to inspect folder contents and extensions, and shutil to move files. It groups common files into Images, Documents, Videos, Music, Archives, Installers, Code and Others. It asks the user to confirm before moving files, renames duplicates rather than overwriting them, and records each result in a live activity log.

The manual checks recorded in the project report covered mixed file types, unknown and missing extensions, duplicate names, empty folders and a larger set of small files. The reported results matched expected behavior, including completing the larger-file check in under a second.

Project detail	Information
Project	File Organizer using Python
Student	Kavya Kishor Jain
Roll number	24EJCCS186
Programme	B.Tech, Semester V, Year III, CSE
Institution	Jaipur Engineering College & Research Centre
ITS Coordinator	Ms. Dhara Rathore, Associate Professor

TABLE OF CONTENTS

1 Preface 4
2 Introduction 4
2.1 Project Context 4
2.2 Objectives 4
2.3 Scope 4
3 Problem Statement 4
4 Existing and Proposed Solution 4
4.1 Existing Approach 4
4.2 Proposed Solution 5
4.3 Value Provided 5
5 Proposed Design and Model 5
5.1 Processing Workflow 5
5.2 File Categories 5
5.3 Interface and Safety 5
6 Performance Test 5
6.1 Test Plan and Cases 5
6.2 Test Procedure 6
6.3 Performance Outcome 6
7 My Learnings 6
8 Future Work Scope 6
References 6

1 Preface

The Industrial Training project gave me an opportunity to apply Python programming to a practical file-management problem. A crowded Downloads or Desktop folder can contain documents, photographs, videos, music and installers together, making files harder to find. The File Organizer automates the repetitive task of sorting these files into clear categories.

The project was planned as a complete application: identify the problem, define safe sorting rules, build a graphical interface, implement file movement, and check the result using sample folders. This helped me connect programming concepts with a real computer task and understand why clear feedback and safeguards matter when software changes files.

I thank Ms. Dhara Rathore, ITS Coordinator and Associate Professor, Department of Computer Science and Engineering, for her guidance and encouragement. I also thank the Director, Principal, department leadership and Jaipur Engineering College & Research Centre for their support and for providing this training opportunity.

My message to juniors and peers is to choose a problem people actually face, keep the first solution focused, and test it carefully with sample data before using it on important files.

2 Introduction

2.1 Project Context

Users often collect different file types in a single folder. Finding a particular item then takes time, while sorting files manually is repetitive. Python is suitable for this task because its standard library can inspect folders, work with file paths, move files and create a simple graphical interface without extra packages.

2.2 Objectives

- Provide a graphical way to choose a folder.
- Identify each loose file by its extension and place it in a matching category folder.
- Ask for confirmation before making changes.
- Avoid replacing files when a duplicate name already exists.
- Show moved and skipped files, plus a final count, in an activity log.
- Apply and document practical Python programming and file-handling skills.

2.3 Scope

The application processes files directly inside the folder selected by the user. It does not search nested subfolders. Categories come from a fixed extension list; unrecognised and extensionless files go to Others.

3 Problem Statement

Files of many types accumulate together in common folders such as Downloads. Users must identify and move them individually, which takes time and can lead to mistakes. This project builds a Python tool that scans a selected folder and sorts its files into category subfolders safely and transparently.

4 Existing and Proposed Solution

4.1 Existing Approach

The basic existing approach is to open a folder, inspect each item and move it manually. This works for a small number of files but becomes tedious as the folder grows and relies on the user remembering where each file belongs.

4.2 Proposed Solution

The File Organizer uses a category dictionary to map known extensions to folder names. It scans the selected folder, creates category folders when needed and moves files to their destinations. A confirmation dialog, duplicate-name handling and per-file error reporting make the operation safer and easier to follow.

4.3 Value Provided

The tool reduces repetitive sorting, applies consistent rules and gives the user a visible record of actions. It uses Python's standard library, so no third-party packages are required.

5 Proposed Design and Model

5.1 Processing Workflow

Stage	What happens
Folder selection	The user chooses a folder using a Browse dialog.
File scanning	The program lists files directly in the chosen folder.
Extension check	Each filename is checked for its extension.
Category decision	The extension is matched to a category; unmatched files go to Others.
Folder creation and move	The destination folder is created if needed and the file is moved.
Log and summary	The interface records every move or skip and reports totals.

5.2 File Categories

Category	Recognised file types
Images	.jpg, .jpeg, .png, .gif, .bmp, .svg, .webp, .heic
Documents	.pdf, .doc, .docx, .txt, .rtf, .odt, .xls, .xlsx, .ppt, .pptx, .csv
Videos	.mp4, .mkv, .mov, .avi, .wmv, .flv, .webm
Music	.mp3, .wav, .flac, .aac, .ogg, .m4a
Archives	.zip, .rar, .7z, .tar, .gz
Installers	.exe, .msi, .dmg, .pkg, .deb
Code	.py, .js, .html, .css, .java, .cpp, .c, .json
Others	Unlisted extension or no extension

5.3 Interface and Safety

The tkinter window includes a folder browse control, an Organize Files button and a scrollable activity log. A confirmation prompt appears before any file is moved. If a destination already contains a file with the same name, the incoming file is renamed with a numeric suffix instead of overwriting the existing one. If a file cannot be moved, the problem is logged so the rest of the batch can continue.

6 Performance Test

The checks recorded in the project report used small sample folders. The results below reflect those manual checks.

6.1 Test Plan and Cases

Test case	Expected behavior	Reported result
Mixed folder with one file from each category	Each file is placed in its matching category folder.	Matched; all seven listed categories sorted correctly.

Unknown and missing extensions	Both files are placed in Others.	Matched.
Duplicate name at destination	Incoming file receives a numeric suffix; existing file remains.	Matched.
Empty folder or only subfolders	Completes without error and reports zero organized files.	Matched.
Many small files	Files are sorted and the log remains readable.	Completed in under one second.

6.2 Test Procedure

Prepare a sample folder with representative files and a destination containing a duplicate filename.

Select the sample folder in the application and confirm the organize action.

Check the created category folders, filenames and activity log against expected results.

Repeat with unknown extensions, no extensions, an empty folder and a larger set of small files.

6.3 Performance Outcome

The documented manual tests matched the expected behavior. The larger sample of small files completed in under a second. These results cover the stated scenarios; they do not establish performance for every folder size, storage device or permission condition.

7 My Learnings

Through this project, I practised using Python modules to interact with the file system, splitting and joining paths, writing loops and conditions, and organising logic into functions. I connected program logic to a tkinter interface and learned to report progress in a way users can understand.

The project highlighted the importance of testing edge cases and protecting user data. Confirmation prompts, duplicate handling and error logging made safety part of the design. Planning the workflow, documenting requirements and recording test outcomes also strengthened my problem-solving and technical communication skills.

8 Future Work Scope

Add optional recursive sorting with safeguards for nested folders.

Let users edit categories and extensions in the graphical interface.

Add an undo option using a record of each file’s original location.

Create automated unit tests for normal and unusual file and permission cases.

Package the application as a standalone executable for users without Python.

Add scheduled runs or a watched-folder option for new downloads.

Prepare cross-platform releases for Windows, macOS and Linux.

References

[1] Python Software Foundation, “Python 3 Documentation,” https://docs.python.org/3/.

[2] Python Software Foundation, “os — Miscellaneous Operating System Interfaces,” https://docs.python.org/3/library/os.html.

[3] Python Software Foundation, “shutil — High-Level File Operations,” https://docs.python.org/3/library/shutil.html.

[4] Python Software Foundation, “tkinter — Python Interface to Tcl/Tk,” https://docs.python.org/3/library/tkinter.html.

[5] GitHub Docs, “About READMEs,” https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes.